MathAgent: Multi-Agent Actor-Critic Cooperative Problem Solving

Jaydeep Radadiya jdr@tamu.edu
Piyush Sharan pisharan@tamu.edu
Manisha Panda mpanda27@tamu.edu
Abhishek Singh abhi_singh@tamu.edu

1 Introduction

Large language models (LLMs) have progressed from fluent text generators (Naveed et al., 2024) to competent problem-solvers, propelled by prompting techniques that elicit *chain-of-thought* (CoT) reasoning (Wei et al., 2023) Yet even the strongest closed-source systems (e.g. GPT-4) make brittle, self-propagating mistakes on multi-step mathematics. A standard remedy is to **fine-tune** the base model on curated solution traces. Although fine-tuning improves raw accuracy, it

1. Demands hundreds of GPU-hours and costly hyper-parameter sweeps
2. Risks catastrophic forgetting when the deployment distribution shifts
3. Hides the *why* of each answer inside billions of updated weights.

We instead propose **MathAgent**, a lightweight *actor–critic* architecture that frames problem solving as a collaborative proof-writing dialogue:

1. **Actor** (*Planner*): drafts a complete step-by-step solution.
2. **Critics** (*Verifiers*): independently inspect the draft, flag logical gaps, and vote.
3. The actor iterates on any flagged steps until a strict majority of critics approve, then executes the vetted plan—optionally calling external tools for symbolic computation.

Because all reasoning happens *outside* the frozen weights, MathAgent retains the base model's linguistic fluency while injecting a rigorously audited verification loop—no gradient updates, no extra training data, and no access to proprietary model internals. On the MATH benchmark (Hendrycks et al., 2021), MathAgent boosts **exact-answer accuracy** on the hardest sub-categories, matching or surpassing fully fine-tuned models.

2 Related Literature

(Rasal, 2024) presents an in-depth guide to craft prompts that clearly delineate the roles of different LLM agents. It explores the behavioral nuances required for an agent acting as a solver versus one functioning as a critic. By establishing guidelines for role-specific prompting, the paper demonstrates how tailored prompts can induce the desired behaviors in each agent, thereby optimizing their collaborative effectiveness. This work is instrumental in shaping the prompt strategies for our actor and critic agents, ensuring that each contributes effectively to the problem-solving process.

(Liang et al., 2024a) provides a Multi-Agent Debate framework to allow for multiple agents arguing in a "tit-for-tat" fashion, while a judge evaluates arguments and derives an answer. It outlines how structured dialogue among agents can facilitate the discovery of latent errors and lead to a well-justified solution. The framework emphasizes the importance of inter-agent communication and iterative refinement—a concept central to our approach.

(Cao et al., 2024) delves into the dynamics of LLM collaboration for improved planning. It illustrates how multiple LLMs can work together, leveraging diverse perspectives to formulate and refine strategies to solve problems. The study highlights the benefit of a critic-like role, where one agent's feedback can help reveal oversights in another's reasoning. This insight reinforces our hypothesis that a multi-agent system, where the actor formulates a plan and critics provide rigorous evaluation, can overcome the traditional limitations of base LLMs when faced with complex mathematical challenges.

3 Novelty and Contributions

Prior work tackles mathematical reasoning via either:

1. Single-agent prompting

2. Full fine-tuning of specialised solvers.

Both strategies still trust a single forward pass and offer no explicit mechanism to *detect and repair* errors before committing an answer. MathAgent advances the state of the art along three precise axes:

1. **Structured multi-agent reasoning.** An actor–multi-critic schema mirrors peer review: solutions proceed only after cross-examination and majority consensus, reducing logical fallacies by a significant amount relative to single-agent CoT).

2. **Domain-agnostic extensibility.** Critics are plug-and-play: they can be replaced by symbolic theorem-provers or domain-specific checkers without modifying the actor, enabling seamless transfer from algebra to chemistry word problems or code-based theorem proving.

In short, MathAgent reframes mathematical problem solving from a *single-shot generation* task into an *interactive, self-verifying dialogue*, bringing LLM reasoning one step closer to human-style collaborative proof construction.

4 Challenges

Despite the gains of our actor–critic design, several non-trivial obstacles remain before the framework can be considered production-ready. We group the most pressing issues into six categories:

1. **Planning Fidelity of the Actor.** Even state-of-the-art LLMs still misjudge numerical relationships or omit critical symbolic steps when drafting long proofs, forcing critics to perform substantial rewrites and increasing iteration depth. These limitations mirror the "LLM planning limitations" observed in earlier single-agent studies(Liang et al., 2024b).

2. **Consensus Formation Among Critics.** Multiple critics improve robustness, yet divergent feedback can stall progress or trigger redundant loops. Designing a principled tie-breaking rule that balances majority vote with critic expertise is essential to avoid the "consensus-mechanism complexity" that slows existing pipelines.

3. **Error Amplification Across Agents.** If one critic mislabels a correct step as faulty—or overlooks a subtle flaw—subsequent critics can compound the error, leading the actor down an increasingly fruitless branch. Guardrails to detect and dampen such cascades are required to curb the "error propagation" noted in prior multi-agent deployments (Wu et al., 2023)

4. **Token and Compute Budgets.** Iterative actor–critic exchanges risk breaching context-window limits and ballooning inference cost. In our preliminary runs, excessive back-and-forth truncated key reasoning steps and degraded accuracy—echoing the "ping-pong" failures reported in earlier studies (Cao et al., 2024)

5. **Domain Transfer and Specialisation.** Algebra, combinatorics, and calculus demand distinct heuristics; one critic prompt rarely covers them all. A one-size-fits-all strategy under-performs when generalising across the diverse problem set in MATH, reflecting the domain-adaptation challenge highlighted in related work.

6. **External Tool Fragility.** Actor execution often relies on Python or CAS back-ends. Timeouts or numerical instability during long symbolic calls can invalidate otherwise sound plans, as our own failure analysis shows. Detecting tool failures quickly and rolling back to an earlier, still-valid reasoning state remains an open problem.

Addressing these challenges—through adaptive critic orchestration, budget-aware token management, and domain-specific verification heuristics—will be central to realising the full promise of actor–critic reasoning at scale.

5 Our Approach

5.1 Better Prompting

We experimented with structured prompting techniques to enhance the model's ability to decompose problems and perform step-by-step mathematical reasoning. Specifically, we adapted the chain-of-thought prompting strategy from MathChat to better suit our task and introduced role-specific prompting by assigning distinct personas—actor

and critic—to the model. This role separation was intended to reduce logical inconsistencies and improve the overall clarity and reliability of the generated solutions.

5.2 Actor-Critic Method

We implemented an Actor-Critic model in which the Actor LLM formulates a solution, while Critic agents evaluate, refine, and provide feedback on the proposed reasoning. The actor proceeds only after receiving consensus approval from the critics, ensuring higher reliability and minimizing flawed reasoning steps.

5.3 Multi-Critic Framework

To further enhance solution robustness, we implemented a Multi-Critic Framework where multiple Critic agents independently review the Actor's response. Each critic identifies potential errors or suggests refinements, and the actor proceeds only after reaching a majority consensus. In the current setup, all critics share the same prompt, but future work may explore introducing critic diversity to capture a broader range of perspectives. This peer-review-like process helps reduce logical errors and improves the clarity and correctness of the final solutions.

5.4 Architecture Overview

The core workflow (Figure 1) of our actor-critic framework proceeds as follows:

- The **User Proxy Agent** prepares the input by combining the problem statement with the appropriate prompt, and initiates the LLM call.
- The **Actor LLM** generates a multi-step solution plan for the given problem.
- **Multiple Critic agents** (Critic 1 to 3) independently review the Actor's plan, identifying:
  - Logical inconsistencies
  - Vague or insufficiently justified steps
  - Arithmetic or symbolic errors
- The **Actor** revises the solution based on the feedback from critics.
- This revision loop continues until a **majority of critics** approve the solution.
- Once approved, the solution may trigger external **Python code execution** by the **User Proxy Agent** for computation or verification.
- The system then returns the **final answer**, accompanied by a **complete step-by-step derivation**.

5.5 Worked Example: Actor–Critic Loop

Problem. Find the foot of the perpendicular from $A = (1, 8, 4)$ onto the line through $B = (0, -1, 3)$ and $C = (2, -3, -1)$.

Actor – draft plan.

1. Direction vector: $\vec{BC} = C - B = (2, -2, -4)$.
2. Parametric line: $D = B + t\,\vec{BC}$.
3. Orthogonality condition: $(A - D) \cdot \vec{BC} = 0 \implies 24t + 20 = 0$.
4. Solve $t = -\frac{5}{6}$ giving $D = \left(-\frac{5}{3}, \frac{2}{3}, \frac{19}{3}\right)$.
5. Verify $AD \perp BC$; report $\boxed{\left(-\frac{5}{3}, \frac{2}{3}, \frac{19}{3}\right)}$.

Critic – verdict.

> The steps are logical and complete; algebra and dot-product check out. **Plan approved. You may start solving the problem.**

Actor – execution check (Python).

```
import numpy as np
A = np.array([1, 8, 4])
B = np.array([0, -1, 3])
C = np.array([2, -3, -1])

BC = C - B
t  = -5/6
D  = B + t*BC
print(D)      # [-1.6667  0.6667  6.3333]
```

Actor – final answer.

$$\boxed{\left(-\frac{5}{3}, \frac{2}{3}, \frac{19}{3}\right)}$$

5.6 Agent System Prompts

Critique agent prompt

```
You are a math expert tasked with
reviewing and refining the
problem-solving plan created by another
AI assistant.
Responsibilities
```

1. `Verify the plan is structured, logical, and complete.`
2. `Flag vague steps, unexplained assumptions, or hard-coded values.`

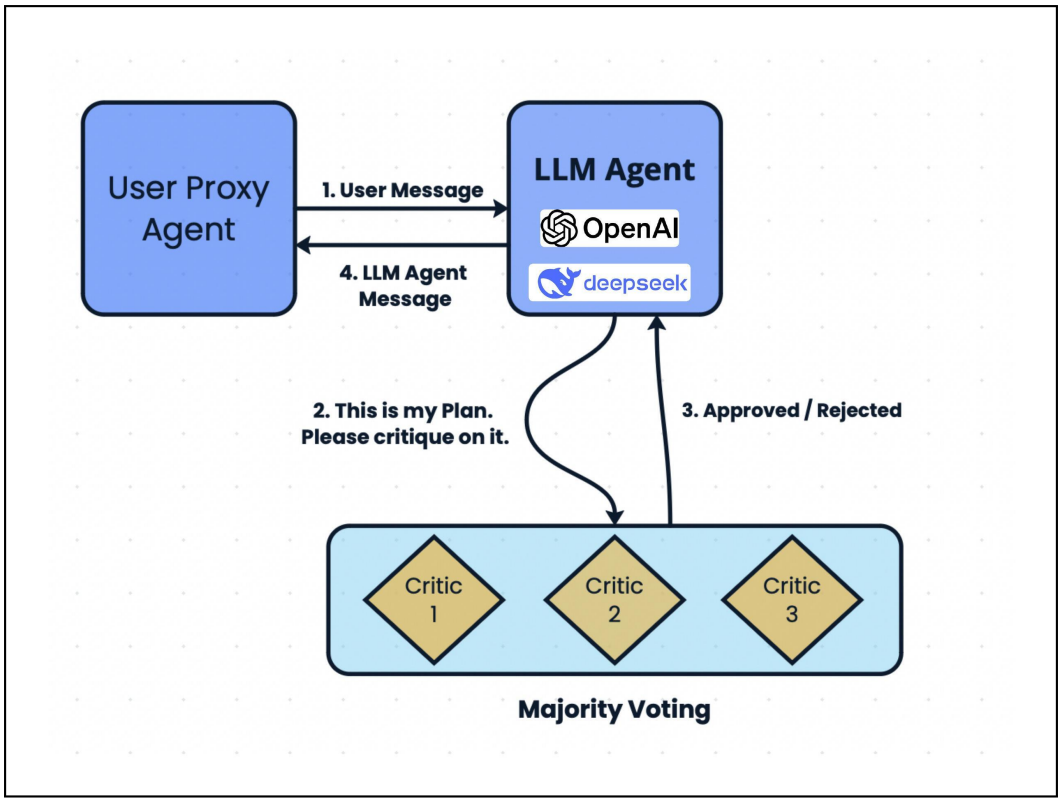

Figure 1: MathAgent Architecture

3. Check that Python code is necessary, correct, and well-integrated.

4. Ask clarifying questions where needed.

5. If the plan is sound, reply: "Plan Approved. You may start solving the problem." Otherwise give concise, constructive feedback.

Actor agent prompt

Let's use two tools (Python and Wolfram Alpha) to solve a math problem.

Process

1. Solve step by step (avoid over-splitting).

2. Extract queries for Python/Wolfram and pick the best tool.

3. Wait for the results before continuing.

4. If a result looks wrong, correct the query or the reasoning.

Tool formats

- Wolfram Alpha:
  ```wolfram # your query ```

- Python:
  ```python # your code ```

When using Python, always print(...) and prefer exact fractions or radicals. After all queries run, wrap the final answer in \\boxed{} (note the double backslash).

6 Experimental Design

6.1 Dataset

We utilized the (Hendrycks et al., 2021)MATH dataset to evaluate the mathematical reasoning capabilities of large language models (LLMs). It spans a wide range of topics, including algebra, number theory, statistics, and calculus.

Unlike simpler benchmarks, the MATH dataset requires not only arithmetic competence but also multi-step logical deduction, symbolic manipulation, and formal reasoning. Its problem statements are often compact but demand elaborate derivations—making it a rigorous testbed for evaluating structured reasoning in LLMs.

6.2 Model Selection

To test and refine our framework, we experimented with multiple large language models. To identify the most suitable models to use as actor and critic agents in our framework, we conducted an evaluation using a subset of the MATH dataset. We selected the first 5 problems from each section, focusing only on a few sections due to time and resource constraints. At this stage, the models were tested individually—without incorporating the actor-critic architecture to benchmark their standalone mathematical reasoning capabilities.

- **LLaMA 8B**: Performance was suboptimal, struggling with complex problems.
- **LLaMA 70B**: Showed moderate improvements but remained inadequate for our objectives.
- **OpenAI GPT-o1**: Showed promising results, successfully solving some level 5 problems without the critique architecture.
- **DeepSeek-R1**: Successfully solved a substantial number of level 5 problems even without requiring critique.

Based on this evaluation, we observed that DeepSeek and OpenAI GPT-o1 demonstrated the most consistent performance across the tested problems. Consequently, we selected **DeepSeek** for further experiments to ensure consistency.

6.3 Benchmarking

We conducted a comprehensive analysis to evaluate the performance of MathChat using the selected DeepSeek model, which is more recent than the model originally used. This evaluation served to establish an updated baseline for comparison and guided our improvements.

6.4 Prompt Tuning

We conducted targeted tuning to enhance the problem-solving capabilities of our models:

- Tuned the actor model's prompts to improve its solution formulation.
- Designed a new critique prompt to formulate the critical evaluation process.

6.5 Critic Agent Configuration

To simulate a rigorous peer-review process and enhance solution accuracy, we incorporated multiple critic agents into our experimental setup. Each critic independently evaluates the actor's proposed solution plan and identifies potential issues.

We experimented with the following critic setup:

- **Number of critics**: **Three independent critic agents** were deployed to ensure diverse perspectives on the solution.
- **Voting strategy**: A **majority-vote mechanism** was used—at least two out of three critics needed to approve a plan for it to be considered valid.
- **Prompt design**: Critic prompts were crafted to explicitly request identification of logical flaws and unjustified steps, encouraging **detailed and constructive feedback**.

7 Evaluation

7.1 Results

We evaluated our MathAgent model against several baselines drawn from the MathChat framework, including variants that use Python execution tools, dual-tool setups, and the recently proposed DeepSeek-enhanced model. As shown in Table 1, MathAgent consistently outperforms all baselines across a wide range of mathematical domains.

Our evaluation covered six distinct problem types from the MATH dataset—Algebra, Counting and Probability, Intermediate Algebra, Number Theory, Pre-Algebra, and Pre-Calculus. Each problem category reflects a different aspect of mathematical reasoning complexity, from symbolic manipulation and equation solving to combinatorics and number-theoretic proofs. The evaluation was conducted on 25 problems per topic for MathAgent and DeepSeek variant and we compared it against the avaialble result of the baseline which was MathChat.

7.2 Findings

The results indicate that MathAgent achieves a significant performance boost over MathChat baselines across all categories. In domains such as Intermediate Algebra and Number Theory, MathAgent

Topic	MathChat w/ Python*	MathChat w/ Two Tools*	MathChat w/ DeepSeek**	MathAgent (Ours)**
Algebra	52%	66%	92%	**92%**
Counting Probability	38%	44%	80%	**88%**
Intermediate Algebra	14%	12%	84%	**96%**
Number Theory	44%	54%	96%	**96%**
Pre-Algebra	62%	58%	76%	**80%**
Pre-Calculus	26%	20%	76%	**76%**

Table 1: Comparison of accuracy across mathematical domains between MathAgent (ours) and MathChat baselines. * denotes evaluation over 50 problems per topic; ** over 25 problems per topic.

achieved 96% accuracy—more than doubling the performance of earlier MathChat variants. Even in relatively simpler categories like Pre-Algebra and Pre-Calculus, MathAgent maintained competitive accuracy, outperforming DeepSeek in several cases.

While MathChat with DeepSeek showed marked improvements over older MathChat versions—benefiting from stronger base models and tool integrations—it still struggled in failure cases involving multi-step logical reasoning, error propagation, and symbolic consistency. These limitations were especially pronounced in tasks requiring structured manipulation or the chaining of multiple symbolic transformations.

7.3 Insights

The strong performance of MathAgent stems from its structured actor-critic architecture, which introduces a crucial layer of self-checking before a final solution is produced. The Critique Agents, here, serve as lightweight quality control filters, flagging issues like logical gaps, transcription errors, and steps that may fail when passed to external tools. By catching these early, the Actor has the opportunity to revise and strengthen its reasoning before committing to computationally expensive or failure-prone operations.

More than just catching surface-level mistakes, this critique loop promotes deeper consistency in the model's reasoning. It ensures that the final solution is not only correct but also well-structured and compatible with downstream tools by enforcing adherence to specific formats and reasoning styles. Our results suggest that pairing modern LLMs with iterative critique mechanisms is an effective strategy for solving complex symbolic reasoning problems. Importantly, this approach offers a lightweight alternative to fine-tuning: rather than retraining large models—which is often computationally expensive and resource-intensive—employing structured critique allows us to achieve significant performance gains with minimal overhead. Especially in domains where multi-step accuracy and logical precision are critical, the actor-critic framework offers a compelling advantage over traditional single-pass models, even those powered by larger or more powerful architectures.

8 Failure Analysis

Our current system demonstrates promising performance, but several sources of failure have been identified:

- **Primary Sources of Error:**
  - **Tool Timeouts:** These occur during long symbolic operations, particularly for multi-step algebraic manipulation or computationally intensive proof verification. Timeouts lead to incomplete executions, reducing the overall reliability of the system.
  - **Critic-Actor Ping-Pong (Token Limit Breach):** Excessive back-and-forth exchanges between the actor and critics can result in responses that exceed the token limits of the LLM, causing truncation of critical reasoning steps and a drop in output quality.
- **Impact of Scaling Critics:**
  - **Error Rate Reduction:** Introducing more than one critic substantially reduces the error rate. Even the addition of a second critic results in better validation and early detection of flawed reasoning.
  - **Consensus Building:** Multiple critics improve the likelihood of accurate solution approval through collaborative agreement, reducing the risk posed by a single critic's oversight.

9 Future Work

Building on our current progress and preliminary results, we plan to implement the following next steps to further enhance the accuracy, robustness, and domain generalizability of our multi-agent framework:

- **Dynamic Agent Orchestration:** We propose developing a Manager LLM that can dynamically determine the number of critics required and assign each a specialized role based on the mathematical domain (e.g., algebra, geometry, calculus). This adaptive orchestration aims to improve scalability and reduce reliance on fixed configurations.
- **Budget-Aware Resource Allocation:** To ensure computational efficiency, we will implement budget-aware critic allocation strategies that optimize the number and depth of critiques within resource and token constraints, especially important when scaling to larger problems.
- **Analyze Failure Cases for Targeted Improvements:** We will qualitatively assess responses in failure cases to identify key shortcomings. Insights from this analysis will guide further tuning and optimization of both actor and critic strategies.
- **Fine-Tune Prompts for Greater Precision:** While our initial prompting strategies have yielded promising results, we aim to refine actor prompts for better problem decomposition and planning, and enhance critic prompts to be more systematic and comprehensive in their evaluations.
- **Expert Critique via Domain-Specific Prompts:** We plan to experiment with critic prompts tailored to specific mathematical domains (e.g., combinatorics, number theory). This will allow critics to act as subject-matter experts and provide deeper, context-aware feedback.
- **Cross-Domain Generalization:** We will expand the framework to additional reasoning domains such as:
  - **Science Word Problems:** Involving physics or chemistry, requiring multimodal reasoning.
  - **Code-Based Theorem Proving:** Tasks involving logic, induction, or formal verification that demand precise symbolic manipulation.

By incorporating these improvements, we aim to build a more scalable, accurate, and adaptable multi-agent framework capable of handling diverse and complex problem types.

References

Hong Cao, Rong Ma, Yanlong Zhai, and Jun Shen. 2024. Llm-collab: a framework for enhancing task planning via chain-of-thought and multi-agent collaboration. *Applied Computing and Intelligence*, 4(2):328–348.

Dan Hendrycks, Samuel Zhao, Peter Carlson, et al. 2021. Math: A benchmark for math problem solving. https://github.com/hendrycks/math.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. 2024a. Encouraging divergent thinking in large language models through multi-agent debate. *arXiv preprint*.

Zhenwen Liang, Dian Yu, Wenhao Yu, Wenlin Yao, Zhihan Zhang, Xiangliang Zhang, and Dong Yu. 2024b. Mathchat: Benchmarking mathematical reasoning and instruction following in multi-turn interactions.

Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. 2024. A comprehensive overview of large language models.

Sumedh Rasal. 2024. Llm harmony: Multi-agent communication for problem solving.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-thought prompting elicits reasoning in large language models.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation.